

Memento Design Pattern

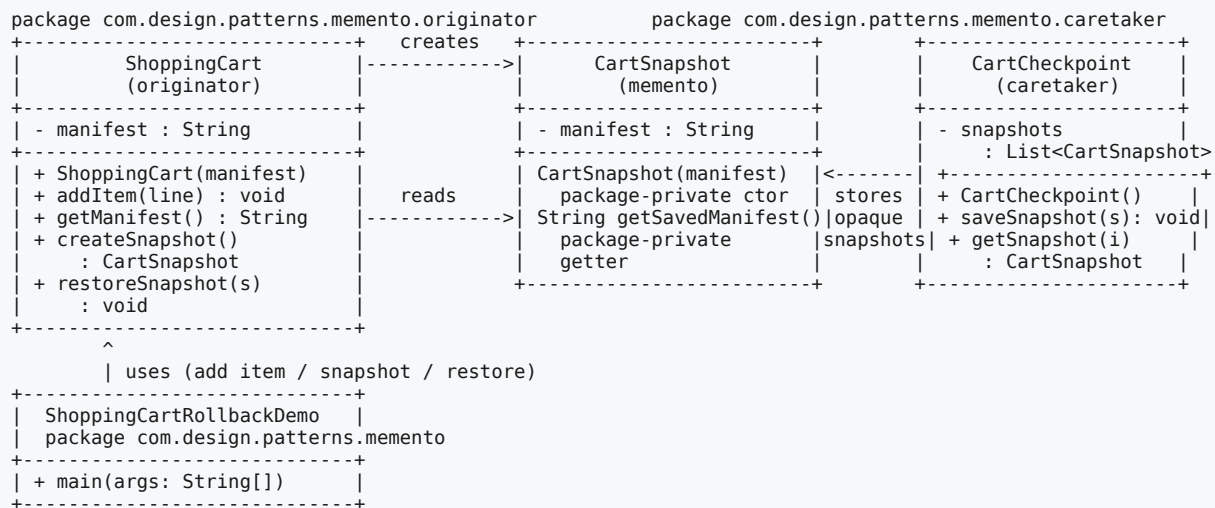
Intent

Capture an object's internal state so it can be **saved and later restored** — without exposing that object's internals to the outside world. In one word: **rollback**. You snapshot an object now, keep the snapshot somewhere, and roll the object back to it later, all while the object's private data stays private to the object that owns it.

Here the object is a `ShoppingCart` that items keep getting added to; each snapshot is a `CartSnapshot`; and a `CartCheckpoint` keeps the snapshots so a risky bulk-add can be undone if it turns out to be a mistake.

This is the **canonical GoF Memento**, implemented with the textbook "wide interface to the originator, narrow interface to the caretaker" trick: the memento's state accessors are **package-private**, and the memento lives in the **same package as its originator** — so only `ShoppingCart` can ever read or write what's inside a `CartSnapshot`. `CartCheckpoint`, sitting in a different package, can hold and hand back snapshots but has no way to look inside one; the compiler enforces this, not just convention.

UML class diagram



Note that `CartSnapshot`'s constructor and `getSavedManifest()` carry **no access modifier**, i.e. package-private/default access. `CartCheckpoint` lives in `com.design.patterns.memento.caretaker` — a different package — so it can declare a field of type `CartSnapshot` and pass the reference around, but it cannot call either member. Only classes in `com.design.patterns.memento.originator` (in practice, only `ShoppingCart`) can.

The players

originator/ShoppingCart	the ORIGINATOR — owns the cart contents, snapshots them, restores from a snapshot
originator/CartSnapshot	the MEMENTO — an immutable, opaque-to-outsiders snapshot of the cart
manifest	
caretaker/CartCheckpoint	the CARETAKER — stores snapshots for later restore, never opens them
ShoppingCartRollbackDemo	the demo — add items, snapshot, do a risky bulk-add, snapshot, then roll back

- **Originator (ShoppingCart)** — creates a snapshot of its own state (`createSnapshot()`) and knows how to restore itself from one (`restoreSnapshot(...)`). It is the only class that ever calls `CartSnapshot`'s constructor or its getter, which is exactly why it lives in the same package as the memento.
- **Memento (CartSnapshot)** — a passive value object holding the saved manifest. Its type is `public` (so it can be named, stored, and passed around by any package), but its constructor and its state-reading method are package-private, so no code outside `com.design.patterns.memento.originator` can ever construct one or read what's inside it.
- **Caretaker (CartCheckpoint)** — holds snapshots for safe-keeping (the rollback stack) and treats them as **opaque**. It never reads or edits the manifest inside — and after this fix, it structurally **cannot**, even if a future maintainer tried, because the compiler would reject the call.

The code, line by line

ShoppingCart — the originator (`originator/ShoppingCart.java`)

```
package com.design.patterns.memento.originator;

public class ShoppingCart {
    private String manifest;

    public ShoppingCart(String manifest) {
        this.manifest = manifest;
    }

    public void addItem(String line) {
        this.manifest += line;
    }

    public String getManifest() {
        return this.manifest;
    }

    public CartSnapshot createSnapshot() {
        return new CartSnapshot(this.manifest);
    }

    public void restoreSnapshot(CartSnapshot snapshot) {
        this.manifest = snapshot.getSavedManifest();
    }
}
```

- `package com.design.patterns.memento.originator;` — places `ShoppingCart` in the `originator` package. This package is now shared with `CartSnapshot`, which is what lets the two collaborate through package-private members while staying closed off to everyone else.
- `public class ShoppingCart {` — the class is `public` so the demo class (a different package, `com.design.patterns.memento`) and the caretaker package can reference the *type*, hold variables of it, and call its public API.
- `private String manifest;` — the state being protected and snapshotted: a running, line-per-item text manifest of the cart's contents. `private` means only `ShoppingCart`'s own methods can read or write it directly; no external code, not even `CartSnapshot`, touches this field.
- `public ShoppingCart(String manifest) { this.manifest = manifest; }` — the constructor seeds the initial manifest, e.g. "2x Espresso Beans 1kg\n" in the demo.
- `public void addItem(String line) { this.manifest += line; }` — the "normal work" method: it appends an item line and mutates `manifest`. Everything this pattern exists to let you roll back happens through calls like this one.
- `public String getManifest() { return this.manifest; }` — a plain, safe read accessor for the current manifest; used by the demo to print the final result after restore.
- `public CartSnapshot createSnapshot() { return new CartSnapshot(this.manifest); }` — the **save** operation. `ShoppingCart` builds a brand-new `CartSnapshot`, handing it the *current* manifest. Because `CartSnapshot`'s constructor is package-private and `ShoppingCart` sits in the same package, this call compiles; nothing outside `originator` could write this line. The originator decides exactly what state gets captured — here, all of it.
- `public void restoreSnapshot(CartSnapshot snapshot) { this.manifest = snapshot.getSavedManifest(); }` — the **rollback** operation. `ShoppingCart` takes a snapshot handed to it (typically fetched from `CartCheckpoint`) and calls `getSavedManifest()` on it — legal only because `ShoppingCart` and `CartSnapshot` share a package. The cart's *entire* state (`manifest`) is overwritten from the snapshot's saved value, so the restore is complete, not partial.

CartSnapshot — the memento (`originator/CartSnapshot.java`)

```
package com.design.patterns.memento.originator;

public class CartSnapshot {
    private String manifest;

    CartSnapshot(String manifest) {
        this.manifest = manifest;
    }

    String getSavedManifest() {
        return this.manifest;
    }
}
```

- `package com.design.patterns.memento.originator;` — the deliberate, load-bearing line of this whole module: putting the memento in the *same package as its originator* is what makes package-private access mean "only

the originator" instead of "only itself."

- `public class CartSnapshot {` — the **class** stays `public`. This is required so that `CartCheckpoint`, living in the `caretaker` package, can still declare a field of type `List<CartSnapshot>` and method signatures like `getSnapshot(int) : CartSnapshot`. Hiding the type entirely was not an option — the caretaker legitimately needs to *hold and pass* the type; it just must never *open* it.
- `private String manifest;` — the frozen copy of the originator's manifest at the moment of the snapshot. `private` even from `ShoppingCart` directly — the only way in or out is through the snapshot's own (package-private) constructor and getter.
- `CartSnapshot(String manifest) { this.manifest = manifest; }` — **no access modifier** = package-private (default access). Only code in `com.design.patterns.memento.originator` can call `new CartSnapshot(...)`. In practice that's only `ShoppingCart.createSnapshot()`. `CartCheckpoint` cannot construct one itself, and nothing in the `caretaker` package could even if it tried — this line would fail to compile there.
- `String getSavedManifest() { return this.manifest; }` — also package-private. Only classes in `originator` (i.e., `ShoppingCart`) can call it. This is the fix for the classic Memento pitfall: a getter like this being `public` would let *any class* — including `CartCheckpoint` — call `snapshot.getSavedManifest()` and read the cart's saved contents without going through `ShoppingCart` at all, silently breaking encapsulation. Restricting it to package-private, combined with placing the memento inside the originator's package, makes that impossible: `CartCheckpoint` (package `caretaker`) referencing `snapshot.getSavedManifest()` is a compile error, not just bad practice.

CartCheckpoint — the caretaker (`caretaker/CartCheckpoint.java`)

```
package com.design.patterns.memento.caretaker;

import java.util.ArrayList;
import java.util.List;

import com.design.patterns.memento.originator.CartSnapshot;

public class CartCheckpoint {

    private List<CartSnapshot> snapshots;

    public CartCheckpoint() {
        this.snapshots = new ArrayList<>();
    }

    public void saveSnapshot(CartSnapshot snapshot) {
        this.snapshots.add(snapshot);
    }

    public CartSnapshot getSnapshot(int index) {
        return this.snapshots.get(index);
    }
}
```

- `package com.design.patterns.memento.caretaker;` — a package distinct from `originator`. This distinction is what gives the package-private access modifiers on `CartSnapshot` real teeth: `CartCheckpoint` is outside the circle of trust.
- `import java.util.ArrayList; import java.util.List;` — standard JDK collection types used to hold the rollback checkpoints.
- `import com.design.patterns.memento.originator.CartSnapshot;` — imports the memento **type** so `CartCheckpoint` can name it in field and method signatures. Importing the type does not grant access to its package-private members — Java access control is enforced per-member, independent of imports.
- `public class CartCheckpoint {` — `public` so the demo class can construct and use it.
- `private List<CartSnapshot> snapshots;` — the checkpoint list: an ordered list of opaque cart snapshots. `CartCheckpoint` never looks inside any element; it only ever adds to, or indexes into, this list.
- `public CartCheckpoint() { this.snapshots = new ArrayList<>(); }` — starts with an empty checkpoint list.
- `public void saveSnapshot(CartSnapshot snapshot) { this.snapshots.add(snapshot); }` — the **save** side of the caretaker's job: append a snapshot handed to it by the client. Notice the parameter is stored whole — `CartCheckpoint` never calls anything on `snapshot` here, and structurally cannot (there is nothing public to call besides object identity methods inherited from `Object`).
- `public CartSnapshot getSnapshot(int index) { return this.snapshots.get(index); }` — the **retrieve** side: hand a stored snapshot back out by index, unopened, for the *caller* (the demo, which then passes it to `ShoppingCart.restoreSnapshot`) to use. `CartCheckpoint` itself never reads what's inside.

ShoppingCartRollbackDemo — the demo / client (`ShoppingCartRollbackDemo.java`)

```
package com.design.patterns.memento;

import com.design.patterns.memento.caretaker.CartCheckpoint;
import com.design.patterns.memento.originator.ShoppingCart;

public class ShoppingCartRollbackDemo {

    public static void main(String[] args) {
        System.out.println("Shopping Cart Bulk-Update Rollback");
        ShoppingCart cart = new ShoppingCart("2x Espresso Beans 1kg\n");
        CartCheckpoint checkpoint = new CartCheckpoint();
    }
}
```

```

        cart.addItem("1x Pour-Over Filter Pack\n");
        checkpoint.saveSnapshot(cart.createSnapshot());

        cart.addItem("40x Clearance Travel Mugs\n");
        checkpoint.saveSnapshot(cart.createSnapshot());

        cart.restoreSnapshot(checkpoint.getSnapshot(0));

        System.out.println(cart.getManifest());
    }
}

```

- `package com.design.patterns.memento;` — the parent package, one level above both `originator` and `caretaker`. The demo is deliberately outside both — it is a third party that plays client, wiring the originator and caretaker together without belonging to either.
- `import com.design.patterns.memento.caretaker.CartCheckpoint;` / `import com.design.patterns.memento.originator.ShoppingCart;` — brings the two public collaborator types into scope by simple name. Neither import grants access to `CartSnapshot`'s internals; the demo also cannot call `getSavedManifest()` directly, and doesn't need to.
- `public class ShoppingCartRollbackDemo {` — `public` and its name matches the filename, as Java requires for a top-level public class. This is the module's entry point.
- `public static void main(String[] args) {` — the standard JVM entry point: `public` so the launcher can call it, `static` so no instance is needed, `void` because nothing is returned, `String[] args` for (unused) command-line arguments.
- `System.out.println("Shopping Cart Bulk-Update Rollback");` — prints a banner line identifying the demo.
- `ShoppingCart cart = new ShoppingCart("2x Espresso Beans 1kg\n");` — creates the originator, seeding its manifest to `"2x Espresso Beans 1kg\n"`.
- `CartCheckpoint checkpoint = new CartCheckpoint();` — creates the caretaker, starting with an empty checkpoint list.
- `cart.addItem("1x Pour-Over Filter Pack\n");` — mutates the originator's state; `manifest` is now `"2x Espresso Beans 1kg\n1x Pour-Over Filter Pack\n"`.
- `checkpoint.saveSnapshot(cart.createSnapshot());` — `cart.createSnapshot()` runs first, packaging the *current* manifest into a new `CartSnapshot` (constructed via the package-private constructor, legal because `createSnapshot()` executes inside `ShoppingCart`, which is in the memento's package). The resulting snapshot is then handed to `checkpoint.saveSnapshot(...)`, which appends it as checkpoint **#0** — taken right before the risky bulk-add below.
- `cart.addItem("40x Clearance Travel Mugs\n");` — the risky bulk-add: mutates the state again; `manifest` is now `"2x Espresso Beans 1kg\n1x Pour-Over Filter Pack\n40x Clearance Travel Mugs\n"`.
- `checkpoint.saveSnapshot(cart.createSnapshot());` — saves a second checkpoint, **#1**, capturing the state including the bulk-added mugs.
- `cart.restoreSnapshot(checkpoint.getSnapshot(0));` — `checkpoint.getSnapshot(0)` returns checkpoint **#0** verbatim, without inspecting it. That reference is passed into `cart.restoreSnapshot(...)`, which (from inside the `originator` package) calls the snapshot's package-private `getSavedManifest()` and overwrites `cart`'s `manifest` with it — rolling the cart back to the state right after the filter pack was added, discarding the accidental 40 clearance mugs.
- `System.out.println(cart.getManifest());` — prints the now-restored manifest.
- Closing braces end `main` and the class.

Why these design decisions

Why not just let callers read/write `manifest` directly for rollback?

Because that would tie every caller to `ShoppingCart`'s internal representation. If rollback meant "save the `String` externally and set it back later," changing `manifest` from a `String` to a structured line-item list would break every caller that ever grabbed the raw state. Memento keeps the *save/restore mechanism* available to the outside world while keeping the *state* itself private: outsiders hold a `CartSnapshot` — a token — and only `ShoppingCart` knows how to fill or read one.

Why three separate roles instead of one class doing everything?

Separation of concerns, and it is the structure of the pattern: - The **originator** (`ShoppingCart`) owns *what* state to save and *how* to restore it — the only class that understands the manifest's meaning. - The **caretaker** (`CartCheckpoint`) owns *when* to save and *which* snapshot to restore — the rollback policy — without needing to understand the manifest at all. - The **memento** (`CartSnapshot`) is the neutral courier between them.

This lets the rollback policy (single-level, multi-level, named checkpoints, branching history) change entirely inside `CartCheckpoint` without touching `ShoppingCart`, and lets `ShoppingCart`'s internals change without touching `CartCheckpoint`.

The encapsulation pitfall this module fixes: package-private accessors + same-package placement

An earlier version of this module had the memento class living in its own separate package, with a **public** constructor and a **public** state-reading getter. That compiles and runs identically, but it is a real encapsulation bug: *any* class anywhere — including the caretaker, or code that has never heard of the originator — could call the getter and read the cart's saved contents directly, or even construct arbitrary memento instances itself. The caretaker was *choosing* not to look, by convention only; nothing stopped it, or any other class, from looking.

The fix applied here: 1. **CartSnapshot lives in `com.design.patterns.memento.originator`** — the same package as `ShoppingCart`. 2. **CartSnapshot's constructor and `getSavedManifest()` carry no public modifier**, leaving them package-private (default access).

Java's default access level means "visible only within the same package." Because `ShoppingCart` shares a package with `CartSnapshot`, it can construct snapshots and read their saved manifest — nothing about the originator's own behavior is compromised. But `CartCheckpoint`, sitting in `com.design.patterns.memento.caretaker`, is structurally locked out: `snapshot.getSavedManifest()` written anywhere in the `caretaker` package (or any other package) is a **compile error**, not a style violation. The class itself (`CartSnapshot`) remains **public** only because `CartCheckpoint` legitimately needs to *name the type* to hold and hand back references — it never needs to open one, and now it provably cannot.

This was verified directly: a throwaway class in a third package (`com.evil.Snoop`) that called `snapshot.getSavedManifest()` on a snapshot obtained from `ShoppingCart.createSnapshot()` failed to compile with error: `getSavedManifest()` is not public in `CartSnapshot`; cannot be accessed from outside package — proving the caretaker (and everyone else outside `originator`) truly cannot inspect a snapshot's contents.

Why does immutability matter here?

The memento must be a **frozen** copy of the state at save time. `String` is immutable in Java, so storing the reference in `CartSnapshot.manifest` is enough — nothing can change it after the fact, including further calls to `cart.addItem(...)`. If `ShoppingCart`'s state were instead a mutable type (`StringBuilder`, `List`, etc.), `createSnapshot()` storing the live reference would be a bug: later mutations to the originator's object would also silently mutate the "frozen" snapshot. The correct fix in that case would be a **defensive copy** inside `createSnapshot()` (e.g., `new StringBuilder(manifest)`). This module is safe without one only because `String` is immutable.

Why does the originator build *and* read the memento, never the caretaker?

Because only the originator understands what its own state means. Letting `ShoppingCart` create the snapshot (`createSnapshot()`) means it controls exactly what gets captured; letting it perform the restore (`restoreSnapshot(...)`) means it controls exactly how state is put back — completely, not partially. `CartCheckpoint` stays deliberately ignorant of both operations, which is what keeps `ShoppingCart`'s internals encapsulated end to end.

Execution flow trace (what happens when `main()` runs)

```
ShoppingCartRollbackDemo.main
├── println("Shopping Cart Bulk-Update Rollback")
├── new ShoppingCart("2x Espresso Beans 1kg\n")    manifest = "2x Espresso Beans 1kg\n"
├── new CartCheckpoint()                          snapshots = []
├── cart.addItem("1x Pour-Over Filter Pack\n")    manifest = "2x Espresso Beans 1kg\n1x Pour-Over Filter Pack\n"
├── cart.createSnapshot()
│   ├── new CartSnapshot(manifest)                (package-private ctor, called from within `originator`)
│   │   └── snapshot #0 { manifest = "2x Espresso Beans 1kg\n1x Pour-Over Filter Pack\n" }
├── checkpoint.saveSnapshot(#0)                  snapshots = [ #0 ]
├── cart.addItem("40x Clearance Travel Mugs\n")   manifest = "2x Espresso Beans 1kg\n1x Pour-Over Filter Pack\n40x Clearance Travel Mugs\n"
├── cart.createSnapshot()
│   ├── new CartSnapshot(manifest)
│   │   └── snapshot #1 { manifest = "2x Espresso Beans 1kg\n1x Pour-Over Filter Pack\n40x Clearance Travel Mugs\n" }
├── checkpoint.saveSnapshot(#1)                  snapshots = [ #0, #1 ]
├── checkpoint.getSnapshot(0)                    returns #0, unopened, unread by CartCheckpoint
├── cart.restoreSnapshot(#0)
│   ├── manifest = #0.getSavedManifest()          (package-private getter, called from within `originator`)
│   └── manifest = "2x Espresso Beans 1kg\n1x Pour-Over Filter Pack\n" ← rolled back, the mugs are discarded
└── println(cart.getManifest())
    └── "2x Espresso Beans 1kg\n1x Pour-Over Filter Pack\n"
```

Snapshot **#1** is still sitting in `checkpoint.snapshots` — it was never removed, so a later `checkpoint.getSnapshot(1)` could still restore it. `CartCheckpoint` never inspected the contents of either snapshot at any point in this trace.

Expected output

Captured from a real run (`JAVA_HOME` pointed at JDK 11):

```
Shopping Cart Bulk-Update Rollback
2x Espresso Beans 1kg
1x Pour-Over Filter Pack
```

The "40x Clearance Travel Mugs" line does not appear — the cart was rolled back to snapshot #0, captured *before* that risky bulk-add ran.

How to run

Spring Boot parent 2.7.7, `java.version` 1.8. Build and run from **inside this module's directory** (not the parent reactor), using JDK 11:

```
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean compile
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 java -cp target/classes
com.design.patterns.memento.ShoppingCartRollbackDemo
```

(Drop `-o` if offline dependency resolution fails and network access is available.)

Relationship to the other Behavioral patterns

- **Memento (this module)** — externalizes and later restores an object's state for rollback, keeping the state encapsulated to the originator.
- **Command** — often paired with Memento: a command executes an action and stores a memento so it can `undo()` back to the prior state.
- **Prototype** (creational) — also copies an object, but to produce a new *usable* instance, not to stash-and-restore one object's own state.
- **State** — represents an object's current mode as an object; Memento snapshots state to *rewind* it rather than to switch behavior.

Reach for Memento when you need **rollback/undo, checkpoints, or history**, and you want to snapshot an object's state without letting any other class — caretaker included — read or write that state directly.